

Maximum Flow Algorithms: Documentation

Kannika Armstrong, Abinava Bharathi Babu, Aqueno Amalraj, Thanvi yadav Sirla

Computer Science and Systems

University of Washington Tacoma

Washington, USA

{akannika, abibabu, aqueno10, thanvys}@uw.edu

How to Run the Code

1. Compiling the Code

Before running the program, you need to compile the Java code. You only need to compile the code when you make changes or updates to the source files. To compile the code, navigate to the directory containing the `.java` files and run the following command:

```
javac *.java
```

This command will compile all Java files in the directory.

2. Running the Code

There are two ways to run the program, depending on how you want to process the input:

- **Option 1: Run a Single Input File**

To run the program with a single input file, use the `runProgram` class with the following command:

```
java -cp . runProgram InputGraph/graphGenerationCode/Bipartite/bg1.txt
```

Replace `InputGraph/graphGenerationCode/Bipartite/bg1.txt` with the actual path to your input file. This will execute the algorithm on the given input and provide the runtime and maximum flow for the chosen algorithm.

- **Option 2: Run with Multiple Input Files**

To run the program with multiple input files, create a `String[]` containing all the input file paths. You can then run the `RunMultipleInputs` class as follows:

```
java -cp . RunMultipleInputs
```

This will execute the algorithm on each of the input files provided in the `String[]` array and output the results for each run.

Outputs and Results

Ensure that you are under the directory that has the java files, in this case, all java files are under the src directory.

```
akannika@Kannikas-MacBook-Pro src % pwd
/Users/akannika/Documents/WT/MS/TCSS543/TCSS543_Project/src
akannika@Kannikas-MacBook-Pro src % ls
Edge.class          InputLib.java      RunMultipleInputs.java
Edge.java           KeyboardReader.class ScalingFordFulkerson.class
FordFulkerson.class KeyboardReader.java ScalingFordFulkerson.java
FordFulkerson.java  PFPPEdge.class     SimpleGraph.class
GraphInput.class    PreFlowPush$Edge.class SimpleGraph.java
GraphInput.java     PreFlowPush$PFPPEdge.class Vertex.class
InputGraph          PreFlowPush.class  Vertex.java
InputLib$InputError.class PreFlowPush.java  runProgram.class
InputLib.class      RunMultipleInputs.class runProgram.java
akannika@Kannikas-MacBook-Pro src %
```

Example1 of how to run the code **Option 1** and the **Output**

Using: java -cp . runProgram InputGraph/graphGenerationCode/Bipartite/bg1.txt

```
akannika@Kannikas-MacBook-Pro src % javac *.java
akannika@Kannikas-MacBook-Pro src % java -cp . runProgram InputGraph/graphGenerationCode/Bipartite/bg1.txt
Ford-Fulkerson Runtime: 298 ms
Ford-Fulkerson Maximum Flow: 26
Scaling Ford-Fulkerson Runtime: 5 ms
Scaling Ford-Fulkerson Maximum Flow: 26
Pre-Flow Push Runtime: 2 ms
Pre-Flow Push Maximum Flow: 26
akannika@Kannikas-MacBook-Pro src %
```

Figure 1 An example of running a program with Bipartite Graph 1 (bg1.txt)

Example2 of how to run the code **Option 2** and the **Output**

Using: java -cp . RunMultipleInputs

```
akannika@Kannikas-MacBook-Pro src % java -cp . RunMultipleInputs

Running for file: InputGraph/graphGenerationCode/Random/rg1.txt
Ford-Fulkerson Runtime: 231 ms
Ford-Fulkerson Maximum Flow: 4
Scaling Ford-Fulkerson Runtime: 0 ms
Scaling Ford-Fulkerson Maximum Flow: 4
Pre-Flow Push Runtime: 0 ms
Pre-Flow Push Maximum Flow: 4

Running for file: InputGraph/graphGenerationCode/Random/rg2.txt
Ford-Fulkerson Runtime: 198 ms
Ford-Fulkerson Maximum Flow: 6
Scaling Ford-Fulkerson Runtime: 1 ms
Scaling Ford-Fulkerson Maximum Flow: 6
Pre-Flow Push Runtime: 3 ms
Pre-Flow Push Maximum Flow: 6

Running for file: InputGraph/graphGenerationCode/Random/rg3.txt
Ford-Fulkerson Runtime: 209 ms
Ford-Fulkerson Maximum Flow: 59
Scaling Ford-Fulkerson Runtime: 3 ms
Scaling Ford-Fulkerson Maximum Flow: 59
Pre-Flow Push Runtime: 0 ms
Pre-Flow Push Maximum Flow: 59

Running for file: InputGraph/graphGenerationCode/Random/rg4.txt
Ford-Fulkerson Runtime: 207 ms
Ford-Fulkerson Maximum Flow: 7
Scaling Ford-Fulkerson Runtime: 1 ms
Scaling Ford-Fulkerson Maximum Flow: 7
Pre-Flow Push Runtime: 9 ms
Pre-Flow Push Maximum Flow: 7

Running for file: InputGraph/graphGenerationCode/Random/rg5.txt
Ford-Fulkerson Runtime: 252 ms
Ford-Fulkerson Maximum Flow: 371
Scaling Ford-Fulkerson Runtime: 5 ms
Scaling Ford-Fulkerson Maximum Flow: 371
Pre-Flow Push Runtime: 0 ms
Pre-Flow Push Maximum Flow: 371
akannika@Kannikas-MacBook-Pro src %
```

Figure 2 An example of running a program with Bipartite Graph 2 (bg2.txt)

Environment and Testing:

- **Operating System:** Tested on MacBook Pro M1 (Apple Silicon) and Windows 11.
- **Java Version:** Java SE 8+ (tested on OpenJDK 17.02.12 and 21.0.1).

Overview of Algorithms

This project implements various algorithms to solve the maximum flow problem in a flow network, including the following:

1. **Ford-Fulkerson Algorithm:**

Uses breadth-first search (BFS) to find augmenting paths and adjusts the residual graph to compute the maximum flow.

2. **Scaling Ford-Fulkerson:**

Optimizes the Ford-Fulkerson algorithm by introducing a scaling parameter to prioritize paths with higher capacities in the initial iterations.

3. **Preflow-Push Algorithm:**

Utilizes a preflow mechanism and pushes flow to the sink by adjusting heights of vertices and excess flow to efficiently find the maximum flow.

Features

- **Maximum Flow Calculation:** Computes the maximum possible flow between a source and sink in a directed graph.
- **Graph Input from File:** Reads the flow network graph from a text-based file containing the edges and their capacities.
- **Residual Graph:** Dynamically updates the capacities of edges as the flow progresses.
- **Multiple Algorithms:** Allows comparisons of different algorithms for flexibility and optimization.

Code Structure

List of Files and Functions

- **FordFulkerson.java**

Implements the classic Ford-Fulkerson algorithm for computing the maximum flow in a network.

- **ScalingFordFulkerson.java**

Implements the Scaling Ford-Fulkerson algorithm, optimizing the basic Ford-Fulkerson approach by introducing a scaling parameter.

- **PreFlowPush.java**
Implements the Preflow-Push algorithm, a more advanced method for maximum flow computation.
- **runProgram.java**
Reads a single input file, runs the chosen algorithm, and outputs the runtime and maximum flow results.
- **RunMultipleInputs.java**
Processes multiple input files listed in a `String[]` and outputs results for each.

Routine Descriptions

1. *FordFulkerson*

Purpose: Find the maximum flow in a network using augmenting paths.

Method: Performs breadth-first search (BFS) to find paths in the residual graph.

Inputs: Source node, sink node, and adjacency matrix of capacities.

Outputs: Maximum flow value.

2. *ScalingFordFulkerson*

Purpose: Optimizes the Ford-Fulkerson algorithm by scaling edge capacities.

Method: Uses a scaling parameter to prioritize high-capacity paths.

Inputs: Source node, sink node, and adjacency matrix of capacities.

Outputs: Maximum flow value.

3. *PreFlowPush*

Purpose: Uses preflows to push excess flow to the sink efficiently.

Method: Maintains a height function to direct flow adjustments.

Inputs: Source node, sink node, and adjacency matrix of capacities.

Outputs: Maximum flow value.

4. *runProgram*

Purpose: Reads a single input file and runs the selected algorithm.

Inputs: File path to the input graph.

Outputs: Runtime and maximum flow value.

5. *RunMultipleInputs*

Purpose: Processes multiple input files and aggregates the results.

Inputs: List of file paths.

Outputs: Runtime and maximum flow value for each file.